MPC-BASED PATH PLANNING AND FOLLOWING FOR AUTONOMOUS VEHICLES
ELEN90026 INTRODUCTION TO OPTIMISATION

SEMESTER 2 2026

1. Introduction

The rapid development of autonomous vehicles has led to an increasing demand for advanced path planning and control algorithms. These algorithms play a critical role in ensuring safe, efficient, and smooth navigation in complex environments. Optimisation techniques and Model Predictive Control (MPC) are widely used in the design of such algorithms due to their ability to handle constraints and optimise performance over a prediction horizon. In this project, students will design MPC-based path planner and follower for an autonomous vehicle operating in a specific environment. In order to successfully implement the MPC controllers in practice, students need to implement a few optimisation algorithms taught in the lectures to solve the MPC problems, in a real-time manner.

2. System Modelling & Model Predictive Control

The accurate and effective mathematical representation of a dynamic system is crucial in the design of MPC. It is also essential to correctly describe key guarantees, to ensure that the control problem can be formulated and approached.

2.1. **Double integrator model.** A double integrator model will be employed to represent the dynamics of the autonomous vehicle. This simple yet effective model captures the essential characteristics of the vehicle's motion, including position and velocity. The model can be represented by the following equations:

$$\begin{aligned} \dot{p}_1 &= v_1,\ \dot{v}_1 = a_1, \\ \dot{p}_2 &= v_2,\ \dot{v}_2 = a_2, \end{aligned} \tag{1}$$

where p_1, p_2 represent the position along p_1 and p_2 axis in the space, v_1, v_2, a_1 and a_2 describe the velocity and acceleration along p_1 and p_2 axis. Considering a continuous system f:

$$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \mathbf{x} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \mathbf{u}, \tag{2}$$

where $\mathbf{x} := [p_1,\ p_2,\ v_1,\ v_2]^T$ and $\mathbf{u} := [a_1,\ a_2]^T$ represent system states and control inputs. Euler's approximation is applied in discretisation for the double integrator model, and can be described as:

$$\mathbf{x}(t+1) = f_d(\mathbf{x}(t), \mathbf{u}(t)) = \begin{bmatrix} 1 & 0 & T_s & 0 \\ 0 & 1 & 0 & T_s \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \mathbf{x}(t) + \begin{bmatrix} \frac{T_s^2}{2} & 0 \\ 0 & \frac{T_s^2}{2} \\ T_s & 0 \\ 0 & T_s \end{bmatrix} \mathbf{u}(t), \tag{3}$$

where f_d is the system representation after discretization, $\mathbf{x}(t)$ and $\mathbf{u}(t)$ are the system state and input at time step $t = 0, 1, \cdots$. T_s is the sampling time. To account for the disturbance, we incorporate additional system noise into the dynamics of the system:

$$\mathbf{x}(t+1) = \begin{bmatrix} 1 & 0 & T_s & 0 \\ 0 & 1 & 0 & T_s \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \mathbf{x}(t) + \begin{bmatrix} \frac{T_s^2}{2} & 0 \\ 0 & \frac{T_s^2}{2} \\ T_s & 0 \\ 0 & T_s \end{bmatrix} \mathbf{u}(t) + \mathbf{w}(t). \tag{4}$$

Considering the constraints imposed by the controller's limitations, the input of the system needs to satisfy the condition: $\mathbf{u}_{\min} \le \mathbf{u}(t) \le \mathbf{u}_{\max}$.

2.2. **Performance Guarantee.** As an example, the track map is given as:

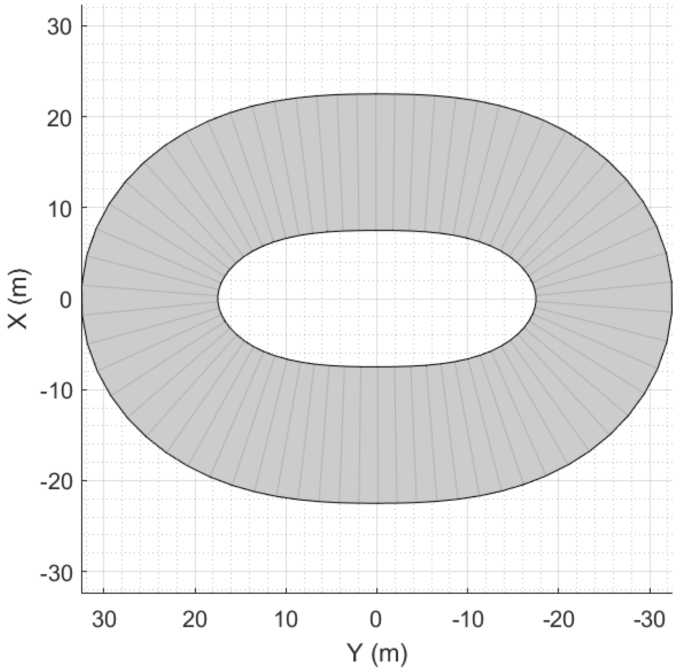

FIGURE 1. Track Map

2.2.1. *Boundary crossing avoidance.* To prevent crossing the boundary, we guarantee that the vehicle's position states lie between two lines that are tangents to the outer and inner bounds of the track for each time step t while also accounting for a safety margin shown in FIGURE 2.

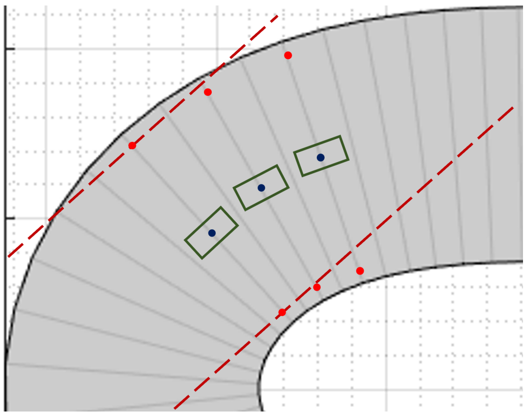

FIGURE 2. Boundary Crossing Avoidance

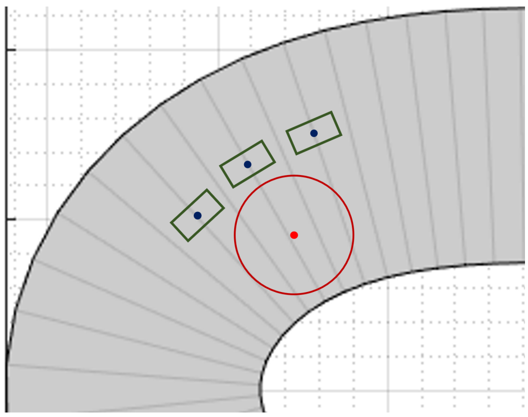

FIGURE 3. Collision Avoidance

2.2.2. *Collision avoidance.* To prevent collisions with obstacles, we begin by modelling the obstacles as circles. Next, we guarantee that the vehicle's position states maintain a safe distance from the centre of each obstacle shown in FIGURE 3. To simplify the handling of collision avoidance constraints, we will adopt a two-step approach. First, we will perform path planning to determine a safe path that avoids collisions with obstacles. Then, we will execute a path-following controller to track the planned path accurately. By separating the tasks of collision avoidance and path following, we can address the constraints effectively and ensure the vehicle's safe navigation.

2.3. **MPC Problem Formulation.** The objective of the Model Predictive Control (MPC) problem is to find the optimal control input sequence that minimises a cost function over a finite prediction horizon while satisfying various constraints. In the path following step, the vehicle should accomplish the following goals: i) minimise the tracking error, ii) minimise the control effort, and iii) avoid crossing the boundary of the track. MPC requires solving an optimisation problem at each time step to determine the optimal control input. It applies the first control input from the optimal sequence to the system and then moves one time step forward. The process is repeated, with the controller recalculating the optimal control sequence at each time step. As such, at time t, the MPC problem can be formulated as:

$$
\begin{aligned}
\min \ & \sum_{i=0}^{N-1} \|\mathbf{x}(i) - \bar{\mathbf{x}}(i)\|_{\mathbf{Q}}^2 + \|\mathbf{u}(i)\|_{\mathbf{R}}^2 + \|\mathbf{x}(N) - \bar{\mathbf{x}}(N)\|_{\mathbf{Q}'}^2, \\
\text{s.t. } & \mathbf{x}(0) = \hat{\mathbf{x}}, \ \mathbf{x}(N) \in \mathcal{X}_f, \\
& \mathbf{x}(i+1) = \mathbf{A} \cdot \mathbf{x}(i) + \mathbf{B} \cdot \mathbf{u}(i) \text{ for all } i = 0, \dots, N-1, \\
& \mathbf{u}_{\min} \le \mathbf{u}(i) \le \mathbf{u}_{\max} \text{ for all } i = 0, \dots, N-1, \\
& d_{\min}(i) \le \mathbf{c}(i)^T \cdot \mathbf{x}(i) \le d_{\max}(i) \text{ for all } i = 0, \dots, N,
\end{aligned} \tag{5}
$$

where $\mathbf{x}(i)$ is the i-th system state from the current state, the i-th reference location and velocity vector at time t:

$$\bar{\mathbf{x}}(i) := [\bar{x}_1(i),\ \bar{x}_2(i),\ \bar{v}_1(i),\ \bar{v}_2(i)]^T \in \mathbb{R}^4,$$

the i-th input solved at time t limited in a range:

$$\mathbf{u}(i) := [a_1(i),\ a_2(i)] \in [\mathbf{u}_{\min}, \mathbf{u}_{\max}],$$

$\mathbf{Q} \in \mathbb{R}^{4\times4}$, $\mathbf{R} \in \mathbb{R}^{2\times2}$ are weighing matrices with positive diagonal entries, $\hat{\mathbf{x}}$ is the estimate or measurement of the current state, and $\mathcal{X}_f$ is a positive control invariant set guaranteeing the feasibility and stability, although it is often neglected in real-world problems. $\mathbf{A} \in \mathbb{R}^{4\times4}$, $\mathbf{B} \in \mathbb{R}^{4\times2}$ are non-negative matrices corresponding to the system dynamics. $d_{\min}(i)$, $\mathbf{c}(i)$ and $d_{\max}(i)$ are given parameters or vectors corresponding to the boundary crossing avoidance shown in FIGURE 2. Later, we drop $\mathcal{X}_f$ and set $\mathbf{Q}' = \mathbf{Q}$ for simplicity.

3. Tasks and Questions

To accomplish the our goal, we break down the process into two parts with multiple tasks. **Part 1** contains **Task 1 - 3** and **Part 2** contains **Task 4 - 5**. In **Task 1**, we begin with a basic task of solving the sequential MPC problems without any state or input constraints. Following this, in **Task 2** and **3**, we iteratively progress to addressing the input and state constraints as formulated in (5). Next, to guarantee collision avoidance, we tackle the path planning task, which requires solving a new MPC problem with additional constraints in **Task 4** for path planning. Finally, the results of the path following (part 1) and planning (part 2) are combined to achieve the desired performance in **Task 5**. Keep your answers to the point and write *two* reports for both parts to get full marks.

Part 1 (62 points)

Task 1 (13 points). Consider the MPC problem without state or input constraints at time t:

$$
\begin{aligned}
\min \ & \sum_{i=0}^{N-1} \|\mathbf{x}(i) - \bar{\mathbf{x}}(i)\|_{\mathbf{Q}}^2 + \|\mathbf{u}(i)\|_{\mathbf{R}}^2 + \|\mathbf{x}(N) - \bar{\mathbf{x}}(N)\|_{\mathbf{Q}}^2, \\
\text{s.t. } & \mathbf{x}(0) = \hat{\mathbf{x}}, \\
& \mathbf{x}(i+1) = \mathbf{A} \cdot \mathbf{x}(i) + \mathbf{B} \cdot \mathbf{u}(i) \text{ for all } i = 0, \dots, N-1,
\end{aligned} \tag{6}
$$

where the equality constraints arise from the system dynamics. We assume that matrices $\mathbf{Q}$ and $\mathbf{R}$ are positive definite and matrices $\mathbf{A}$ and $\mathbf{B}$ are in the form as shown in (3).

Question 1 (5 points). Find the unconstrained condensed QP form of the MPC problem (6).

Hint: With decision vector $\mathbf{z} = \left[\mathbf{u}(0)^T, \ldots, \mathbf{u}(N-1)^T\right]^T$, the general unconstrained QP takes the following form:

$$\min_{\mathbf{z}} \frac{1}{2}\mathbf{z}^T\mathbf{H}\mathbf{z} + \mathbf{h}^T\mathbf{z}.$$

Question 2 (5 points). Is your unconstrained condensed QP a convex optimisation problem? If so, is it strictly convex? Also state your reasons.

Question 3 (3 points). Find the optimality conditions according to your unconstrained condensed QP.

Task 2 (22 points). In this task, our objective is to ensure that our vehicles track the center points of the track shown in FIGURE 1. Consider the optimisation problem only with the control input constraints:

$$\begin{aligned}
\min\ & \sum_{i=0}^{N-1} \|\mathbf{x}(i) - \bar{\mathbf{x}}(i)\|_{\mathbf{Q}}^2 + \|\mathbf{u}(i)\|_{\mathbf{R}}^2 + \|\mathbf{x}(N) - \bar{\mathbf{x}}(N)\|_{\mathbf{Q}}^2, \\
\text{s.t. } & \mathbf{x}(0) = \hat{\mathbf{x}}, \\
& \mathbf{x}(i+1) = \mathbf{A}\cdot\mathbf{x}(i) + \mathbf{B}\cdot\mathbf{u}(i) \text{ for all } i = 0, \ldots, N-1, \\
& \mathbf{u}_{\min} \le \mathbf{u}(i) \le \mathbf{u}_{\max} \text{ for all } i = 0, \ldots, N-1,
\end{aligned} \tag{7}$$

where the input box constraints are originated from controller limitations. We assume that matrices $\mathbf{Q}$ and $\mathbf{R}$ are positive definite and matrices $\mathbf{A}$ and $\mathbf{B}$ are in the form as shown in (3).

Question 1 (1 points). Find the condensed QP form of the optimisation problem (7) with only box constraints.

Hint: With decision vector $\mathbf{z} = \left[\mathbf{u}(0)^T, \ldots, \mathbf{u}(N-1)^T\right]^T$, the QP with only box constraints takes the following form:

$$\begin{aligned}
\min_{\mathbf{z}}\ & \frac{1}{2}\mathbf{z}^T\mathbf{H}\mathbf{z} + \mathbf{h}^T\mathbf{z}, \\
\text{s.t. } & \mathbf{z}_{lb} \le \mathbf{z} \le \mathbf{z}_{ub}.
\end{aligned}$$

Question 2 (6 points). Write the pseudocode to solve your constrained condensed QP by primal projected gradient-based methods. What is the convergence rate of your algorithm? What is the feasible range for choosing the step-size? Also state your reason.

Question 3 (15 points). File `Task2_Question3` provides an example of path following along the given path of a circular track.

- Optimisation algorithm implementation
  - Formulate the MPC problem and the resulting constrained condensed QP.
  - Use `quadprog` to solve the condensed QP problems for $t = 0$. You can also try with other solvers, such as `Gurobi` and `Mosek`.
  - Implement your algorithm to solve the QP problem. Compare its solutions with the ones given by commercial solvers, i.e., `quadprog`, by plotting the deviation.
- MPC controller implementation
  - Use the noisy dynamic model (4) when you update the states.
  - Solve the MPC problem online using the optimisation algorithm implemented in the previous sub-step.
  - Run the simulation and get your car to follow the given path.
  - Plot the trajectory by your algorithm.

Task 3 (27 points). In this task, our objective is to ensure that our vehicles track the center points of the track while avoiding crossing the augmented boundaries shown in FIGURE 4. The augmented boundaries are designed with safety margins to keep the vehicles safely away from the original boundary. Consider the MPC problem:

$$\begin{aligned} \min \ & \sum_{i=0}^{N-1} \|\mathbf{x}(i) - \bar{\mathbf{x}}(i)\|_{\mathbf{Q}}^2 + \|\mathbf{u}(i)\|_{\mathbf{R}}^2 + \|\mathbf{x}(N) - \bar{\mathbf{x}}(N)\|_{\mathbf{Q}}^2, \\ \text{s.t. } & \mathbf{x}(0) = \hat{\mathbf{x}}, \\ & \mathbf{x}(i+1) = \mathbf{A} \cdot \mathbf{x}(i) + \mathbf{B} \cdot \mathbf{u}(i) \text{ for all } i = 0, \dots, N-1, \\ & \mathbf{u}_{\min} \leq \mathbf{u}(i) \leq \mathbf{u}_{\max} \text{ for all } i = 0, \dots, N-1, \\ & d_{\min}(i) \leq \mathbf{c}(i)^T \cdot \mathbf{x}(i) \leq d_{\max}(i) \text{ for all } i = 0, \dots, N, \end{aligned} \tag{8}$$

where the state box constraints prevent the vehicle from crossing the boundaries. We assume that matrices $\mathbf{Q}$ and $\mathbf{R}$ are positive definite and matrices $\mathbf{A}$ and $\mathbf{B}$ are in the form as shown in (3).

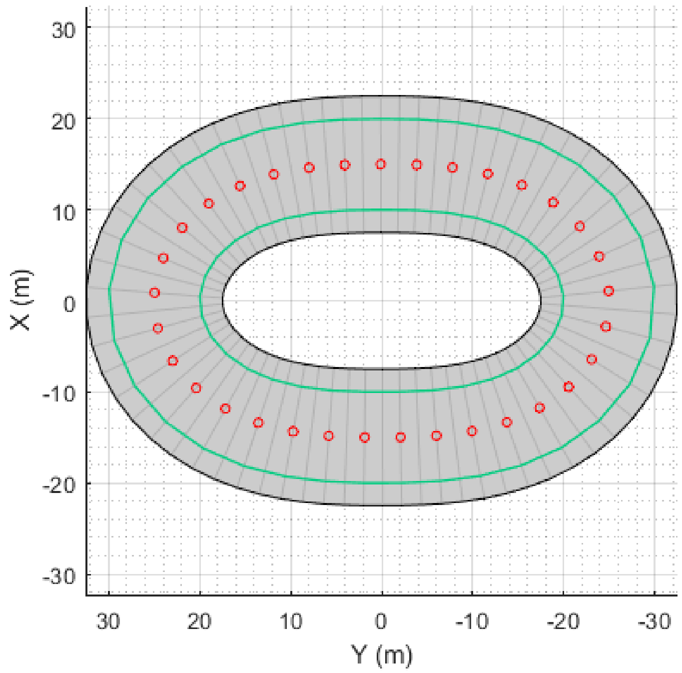

FIGURE 4. Track map (red circle dots for centre path, green curve for augmented boundaries with safety guarantees)

Question 1 (4 points). Find the condensed QP form of the optimisation problem (8) with inequality constraints and box constraints.

Hint: The QP with inequality constraints and box constraints takes the following form:

$$\begin{aligned} \min_{\mathbf{z}} \ & \frac{1}{2}\mathbf{z}^T\mathbf{H}\mathbf{z} + \mathbf{h}^T\mathbf{z}, \\ \text{s.t. } & \mathbf{G}\mathbf{z} \leq \mathbf{g}. \end{aligned}$$

Question 2 (5 points). Find the KKT conditions according to your constrained condensed QP. Does strong duality hold? Also state your reasons.

Question 3 (3 points). In order to have simple projections, find the dual problem of your constrained condensed QP and write the pseudocode code of the gradient method on the dual.

Question 4 (15 points). File `Task3_Question4` provides an example of path following along the given path of a circular track.

- Optimisation algorithm implementation
  - Formulate the MPC problem and the resulting constrained condensed QP.
  - Use `quadprog` to solve the condensed QP problems for $t = 0$. You can also try with other solvers, such as `Gurobi` and `Mosek`.
  - Implement your algorithm to solve the QP problem. Compare its solutions with the ones given by commercial solvers, i.e., `quadprog`, by plotting the deviation.
- MPC
  - Use the noisy dynamic model (4) when you update the states.
  - Solve the MPC problem online using the optimisation algorithm implemented in the previous sub-step.
  - Run the simulation and get your car to follow the given path.
  - Plot the trajectory by your algorithm.